OPERATING SYSTEMS / CORE SUMMARY

운영체제 핵심 통합 요약

CPU 가상화부터 가상 메모리·Linux 보충 자료까지

> **전체를 관통하는 질문**
> 여러 프로그램이 한 컴퓨터를 안전하고 효율적으로 공유하면서, 각자 CPU와 메모리를 가진 것처럼 실행되게 하려면 무엇이 필요한가?

운영체제의 세 가지 큰 주제

주제	핵심 역할	이 파일의 범위
가상화	물리 자원을 쓰기 쉬운 추상화로 제공	CPU·프로세스·메모리가 중심
병행성	실행 순서가 섞여도 공유 데이터를 정확히 관리	개요와 멀티 CPU 동기화
영속성	전원이 꺼져도 파일·데이터를 유지	개요에서만 소개

먼저 기억할 연결 관계

- **프로세스**는 실행 중인 프로그램이다. 스케줄러는 실행 대상을 고르고, 문맥 교환은 그 선택을 실제로 수행한다.
- **주소 공간**은 프로세스가 보는 메모리다. MMU와 페이지 테이블은 가상 주소를 물리 주소로 바꾸고 접근 권한을 검사한다.
- **TLB**는 주소 변환을 빠르게 하고, **스왑과 페이지 교체**는 RAM에 모든 페이지가 들어가지 않을 때를 처리한다.

설계 원칙

정책(policy)은 "누구를 실행할까, 무엇을 내보낼까"를 정한다. **메커니즘(mechanism)**은 "어떻게 전환·변환·전송할까"를 수행한다. 둘을 분리하면 정책을 바꿔도 공통 구현을 재사용할 수 있다.

목표는 편리한 추상화, 낮은 시간·공간 비용, 보호·격리, 신뢰성이다. 응답성·공정성·처리량, 시간·공간, 성능·보안 사이에는 절충이 있다.

구성: 20쪽. 장황한 예제·역사·참고문헌 목록·숙제 전문은 생략하고, 개념·동작 규칙·비교·계산에 집중했다.

02 / 핵심 개념

원본 범위와 읽기 지도

장 번호는 첨부 파일의 북마크 번호를 기준으로 통일

원본 장	주제	원본 PDF 쪽	요약 쪽
02	운영체제 개요	1-17	1
04	프로세스	18-30	3
05	프로세스 API	31-41	3
06	제한적 직접 실행	42-57	4
07	스케줄링 개요	58-71	5
08	멀티 레벨 피드백 큐	72-83	6
09	비례 배분·CFS 보충	84-97	7
10	멀티프로세서 스케줄링	98-111	8
13	주소 공간	112-122	9
14	메모리 API	123-133	9
15	주소 변환·base/bound	134-147	10
16	세그멘테이션	148-159	10
17	빈 공간 관리	160-176	11
18	페이징	177-190	12
19	TLB	191-208	13
20	작은 페이지 테이블	209-223	14
21	스왑 메커니즘	224-235	15
22	페이지 교체·AMAT 보충	236-257	16·19
23	VAX/VMS·Linux 보충	258-280	17-18

사용 자료: 첨부 OS_merged_chapters_02-23.pdf, 총 280쪽. 『운영체제: 아주 쉬운 세 가지 이야기』 한국어 본문과 첨부된 영문판 보충 번역을 통합했다.

쪽수 기준: 원본 PDF의 첫 페이지를 1쪽으로 센다. 원본 본문에 인쇄된 쪽수와는 다를 수 있다. 03·11·12장은 이 파일에 없으므로 별도 내용을 추가하지 않았다.

판본 기준: 한국어 구판과 영문판 보충 번역이 함께 들어 있다. CFS·Linux는 첨부 판본의 설명으로 요약했으며, 현행 구현을 새로 조사한 문서는 아니다. AMAT는 원본 255쪽의 개정 식과 검산을 따른다.

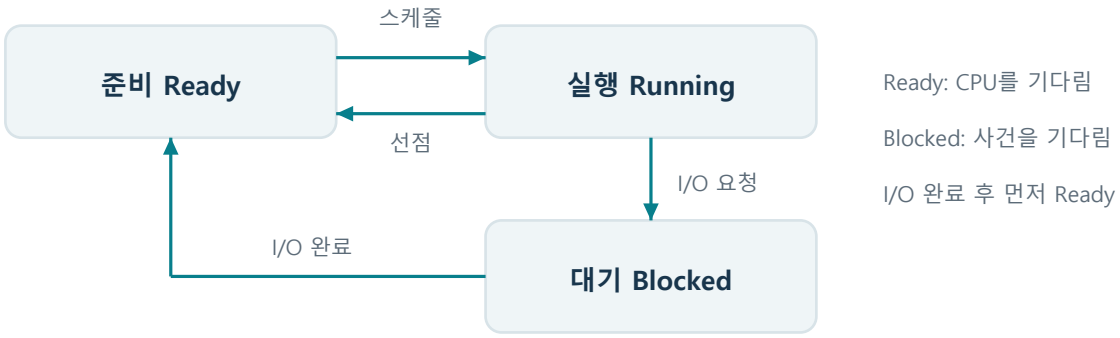
03 / 핵심 개념

프로세스와 Unix API

04·05장 | 실행 상태를 관리하고 새 프로그램을 시작하는 방법

프로세스 = 프로그램 + 실행 상태. 주소 공간, PC·레지스터·스택 포인터, 열린 파일과 I/O 정보 등을 포함한다. 운영체제는 PCB(Process Control Block)에 PID, 상태, 저장된 문맥 등을 관리한다.

프로세스 상태 전이



프로세스 생성 시 코드·데이터를 탑재하고 스택·힙·인자·파일 디스크립터를 준비한다. 실제 탑재는 요구 페이징으로 필요한 시점까지 늦출 수 있다.

fork·exec·wait의 역할

API	핵심 동작	반드시 구분할 점
fork()	부모를 바탕으로 자식 프로세스 생성	부모: 자식 PID / 자식: 0 / 실패: -1
exec 계열	현재 프로세스의 프로그램 이미지를 교체	새 PID를 만들지 않음. 성공하면 기존 코드로 돌아오지 않음
wait()/waitpid()	자식의 상태 변화·종료를 기다리고 종료 정보 회수	부모·자식의 출력 순서는 대기 위치에 따라 달라짐

셸의 기본 흐름
명령 읽기 → fork → 자식의 입출력 환경 설정 → exec
부모 셸은 wait 후 다음 명령을 받는다. fork와 exec 사이에 리다이렉션·파이프 연결을 설정할 수 있다.

fork 후 부모·자식의 실행 순서는 기본적으로 비결정적이다. 주소 공간은 논리적으로 분리되며, 실제 복사는 COW로 늦출 수 있다(17쪽).

04 / 핵심 개념

제한적 직접 실행

06장 | 일반 실행은 빠르게, 특권 작업과 CPU 회수는 통제

> **Limited Direct Execution, LDE**
> 프로그램은 CPU에서 직접 실행한다. 하드웨어의 모드·트랩·인터럽트 지원으로 운영체제가 제어권을 유지한다.

두 가지 해결 과제

과제	해결 장치	효과
위험한 연산 제한	사용자 모드 / 커널 모드	사용자 코드의 특권 명령 실행·커널 메모리 접근 제한
필요한 특권 서비스 제공	시스템 콜과 trap	정해진 진입점에서 인자·권한을 검사하고 요청 수행
CPU 제어권 회수	타이머 인터럽트	프로세스가 양보하지 않아도 운영체제가 선점 가능

시스템 콜의 경로

사용자 코드 → 라이브러리의 호출 래퍼 → 시스템 콜 번호·인자 준비 → trap → 커널 처리 → return-from-trap → 사용자 실행 재개.

트랩 테이블은 부팅 시 운영체제가 설정한다. 사용자 프로그램은 임의의 커널 주소로 진입할 수 없다. 하드웨어는 복귀에 필요한 상태를 저장하고 모드를 바꾼다.

문맥 교환의 경로

타이머 또는 다른 사건으로 커널 진입 → 스케줄러가 다음 프로세스 선택 → 현재 레지스터·커널 스택 문맥 저장 → 다음 문맥 복원 → 다음 프로세스의 사용자 코드로 복귀.

구분	의미
모드 전환	사용자 모드와 커널 모드 사이 이동. 같은 프로세스로 돌아올 수 있음
문맥 교환	실행 프로세스를 바꾸고 그 프로세스의 저장된 문맥으로 전환
협조 방식	프로세스의 시스템 콜·양보에 의존
비협조 방식	타이머 인터럽트로 제어권을 강제로 회수

인터럽트가 발생했다고 항상 다른 프로세스로 바뀌는 것은 아니다. 실행 대상을 고르는 정책과 전환을 수행하는 메커니즘을 구분한다.

05 / 핵심 개념

CPU 스케줄링 기본

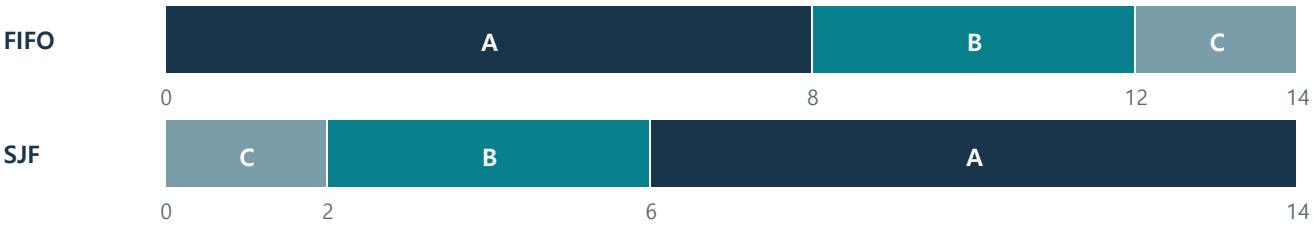
07장 | 어떤 성능 지표를 줄이려는지부터 정한다

> **반환 시간 = 완료 시각 - 도착 시각**
> **응답 시간 = 첫 실행 시각 - 도착 시각**

정책	선택 기준	장점·한계
FIFO / FCFS	먼저 도착한 작업	단순. 긴 작업 뒤 짧은 작업이 기다리는 convoy effect
SJF	가장 짧은 작업; 비선점	동시 도착·실행시간을 아는 조건에서 평균 반환 시간 최적
STCF / SRTF	남은 실행시간 최소; 선점	새 짧은 작업에 즉시 대응. 남은 시간 정보가 필요
RR	준비 큐를 퀀텀 단위로 순환	초기 응답·공정성에 유리. 문맥 교환 비용과 반환 시간 절충

짧은 계산 예제

A=8, B=4, C=2가 시각 0에 도착하고 교환 비용은 0이라고 하자. FIFO의 도착 순서는 A·B·C로 정한다.



FIFO 평균 반환 시간 = (8+12+14)/3 = **11.33**.
SJF 평균 반환 시간 = (2+6+14)/3 = **7.33**.

퀀텀과 I/O의 영향

- 퀀텀이 짧으면 응답은 빨라지지만 교환 횟수가 늘어난다. 길면 교환 비용은 줄지만 RR이 FIFO에 가까워진다.
- I/O로 대기한 작업 대신 다른 준비 작업을 실행하면 CPU 계산과 I/O를 겹쳐 이용률을 높일 수 있다.

SJF·STCF의 최적성은 교환 비용·I/O·실행시간 정보에 관한 가정에 의존한다. 실제 시스템에서는 작업 길이를 모르므로 관측 기반 정책이 필요하다.

06 / 핵심 개념

멀티 레벨 피드백 큐

08장 | 실행시간을 몰라도 관측한 행동으로 우선순위를 조정

> **MLFQ의 목표**
> 짧은 작업과 대화형 작업에 빠르게 응답하면서, 긴 CPU 작업도 계속 진행하게 한다. 과거 CPU 사용으로 앞으로의 행동을 추정한다.

최종 규칙 다섯 가지

규칙	동작
1	우선순위가 높은 큐의 작업을 먼저 실행한다.
2	같은 우선순위의 작업끼리는 RR로 실행한다.
3	새 작업은 가장 높은 우선순위 큐에서 시작한다.
4	현재 단계의 누적 CPU 할당량을 소진하면 아래 큐로 강등한다. I/O 양보 횟수와 무관하다.
5	주기 S마다 모든 작업을 최상위 큐로 올린다(boost).

왜 규칙 4와 5가 필요한가

문제	보완	결과
기아: 높은 큐가 계속 바쁨	주기적인 우선순위 상승	낮은 큐의 작업도 CPU를 얻음
작업 성격이 달라짐	boost 후 다시 관측	CPU 중심에서 대화형으로 바뀐 작업을 재평가
퀀텀 직전에 I/O로 양보해 강등 회피	단계별 누적 CPU 시간 측정	짧게 쪼개 사용해도 할당량 소진 시 강등

높은 큐는 짧은 퀀텀으로 응답성을, 낮은 큐는 긴 퀀텀으로 교환 비용 감소를 노릴 수 있다. 큐 수·할당량·boost 주기는 워크로드에 맞춰 조정한다.

퀀텀은 한 번에 실행하는 길이이고 **단계별 할당량**은 강등 전 사용할 수 있는 누적 CPU 시간이다. 문제에서 둘의 값을 별도로 주면 구분해서 계산한다.

07 / 핵심 개념

비례 배분과 CFS

09장 + 보충 | 처리시간보다 CPU 몫의 공정한 배분에 초점

정책	핵심 규칙	특징
Lottery	실행 가능한 작업의 티켓 중 하나를 무작위 추첨	장기 평균은 티켓 비율에 접근. 짧은 구간의 몫은 흔들림
Stride	가장 작은 pass를 선택하고 실행 후 stride를 더함	결정적으로 비율에 접근. 새 작업의 pass 초기화 주의
CFS	가중치를 반영한 vruntime이 작은 작업을 선택	공정성과 교환 비용을 균형 있게 조절

Lottery의 선택 확률 = 내 티켓 / 전체 실행 가능 티켓
Stride: stride = 큰 상수 K / 티켓 수
실행 후 pass = pass + stride

티켓은 사용자별 화폐로 변환하거나, 서버에 일을 맡길 때 전달할 수 있다. Stride에서 티켓 100·50·250과 K=10,000이면 보폭은 100·200·40으로, 작은 보폭의 작업이 더 자주 선택된다.

첨부 판본의 CFS 핵심

- nice가 작을수록 가중치가 크다. 가중치가 큰 작업은 같은 실제 실행시간에도 vruntime이 더 천천히 증가해 더 많은 CPU를 얻는다.

time_slice(i) = sched_latency × weight(i) / Σweight
Δvruntime(i) = 실제 실행시간 × 1024 / weight(i)

- 최소 실행 길이(min_granularity)로 과도한 교환을 막는다. 위 타임 슬라이스 식은 기본 배분 모델이다.
- vruntime 순서의 레드-블랙 트리로 삽입·삭제를 O(log n)에 처리한다. 잠든 작업의 복귀 시 vruntime을 조정해 CPU 독점을 막는다.

nice=-5의 가중치 3121과 nice=0의 1024를 비교하면 CPU 몫은 약 3:1이다. CFS 설명과 매개변수는 원본 보충의 모델을 따른다.

08 / 핵심 개념

멀티프로세서 스케줄링

10장 | 부하 균형·캐시 친화성·동기화를 함께 고려

CPU가 여러 개면 실행 대상을 고르는 것뿐 아니라 "어느 CPU에 배치할까"가 중요하다. 같은 프로그램이 한 CPU에 다시 실행되면 캐시·TLB에 남은 정보를 재사용할 수 있다.

먼저 구분할 세 가지

개념	역할
캐시 일관성(coherence)	여러 캐시의 같은 메모리 데이터가 모순되지 않도록 하드웨어가 관리
캐시 친화성(affinity)	같은 CPU에서 재실행해 캐시·TLB 재사용률을 높이는 성능 전략
동기화	공유 큐 등 자료 구조의 복합 연산을 락 등으로 보호

단일 큐와 CPU별 큐

구성	장점	단점·보완
SQMS: 공유 큐 하나	구현이 단순하고 유휴 CPU가 작업을 가져가기 쉬움	공유 락 경쟁, 확장성 저하, 친화성 관리 필요
MQMS: CPU별 큐	락 경쟁 감소, 높은 친화성, 확장에 유리	한 CPU만 바쁘고 다른 CPU가 놀 수 있음. 이주로 부하 균형

> **Work stealing**
> 한 CPU가 다른 CPU의 큐를 살펴 작업을 가져온다. 너무 자주 살피면 동기화 비용이 커지고, 너무 드물게 살피면 부하 불균형이 오래 남는다.

판단 순서

- 준비 작업이 충분한데 유휴 CPU가 있으면 큐 분배와 이주 정책을 확인한다.
- CPU를 옮겼을 때 처리속도가 떨어지면 캐시·TLB 재가열 비용과 작업 집합 크기를 확인한다.
- CPU 수를 늘려도 성능이 늘지 않으면 공유 락 경쟁과 메모리 병목을 확인한다.

캐시 일관성이 유지되어도 읽기-수정-쓰기 같은 여러 단계의 공유 자료 갱신에는 별도의 동기화가 필요하다.

09 / 핵심 개념

주소 공간과 메모리 API

13·14장 | 프로그램이 보는 메모리와 할당 라이브러리의 역할

> **주소 공간(address space)**은 프로세스가 보는 가상 메모리의 모습이다.
> 목표: 투명성(프로그램이 물리 배치를 몰라도 됨), 효율성, 보호·격리.

영역	내용	수명·관리
코드·정적 데이터	명령어, 전역·정적 변수	프로그램 이미지의 일부
스택	함수 호출 문맥, 지역 변수, 복귀 주소	함수 호출·복귀에 따라 자동 관리
힙	동적으로 크기가 변하는 데이터	malloc/free로 명시적 할당·해제

서로 다른 프로세스의 같은 가상 주소는 서로 다른 물리 주소에 매핑될 수 있다. 힙과 스택이 서로 반대 방향으로 자라는 그림은 설명용 관례이며, 실제 배치는 더 복잡할 수 있다.

사용할 API와 흔한 오류

API·오류	핵심
malloc(n)	n바이트 요청. 성공 시 포인터, 실패 시 NULL. 내용은 자동으로 0이 되지 않음
calloc / realloc	0으로 초기화 / 크기 변경. realloc 결과가 다른 주소일 수 있음
free(p)	할당 라이브러리가 반환한 블록을 해제. 해제 후 접근·중복 해제 금지
크기·초기화 오류	sizeof(포인터)와 대상 크기 혼동, 문자열 끝의 널 바이트 누락, 미초기화 읽기
수명 오류	메모리 누수, use-after-free, double free, 잘못된 포인터 해제

malloc/free는 라이브러리 함수다. 할당기는 기존 힙 공간을 재사용하고, 필요하면 brk/sbrk나 mmap 등으로 OS에 공간을 요청한다. malloc 호출마다 반드시 시스템 콜이 발생하는 것은 아니다.

지역 포인터 변수는 스택에 있어도, 그 포인터가 가리키는 malloc 블록은 힙에 있다. 할당 위치와 포인터 변수의 위치를 구분한다.

10 / 핵심 개념

주소 변환과 세그멘테이션

15·16장 | 단순한 base/bound에서 영역별 관리로 확장

동적 재배치: base와 bound

> **합법적인 가상 주소: 0 ≤ VA < bound**
> **물리 주소: PA = base + VA**

MMU가 모든 메모리 참조에 대해 범위를 검사하고 주소를 변환한다. base/bound 변경은 특권 연산이다. 운영체제는 생성·종료 때 물리 공간을 관리하고, 문맥 교환 때 레지스터를 저장·복원한다.

예: base=32 KiB, bound=16 KiB, VA=1 KiB이면 PA=33 KiB. VA=16 KiB는 경계 밖이다. 여기서는 bound가 주소 공간의 **크기**를 뜻한다고 가정한다.

세그멘테이션: 논리 영역마다 다른 base/bound

코드·힙·스택을 별도 세그먼트로 나눈다. 사용하지 않는 큰 빈 주소 영역을 RAM에 할당하지 않아도 된다. 세그먼트별 크기·성장 방향·읽기/쓰기/실행 권한을 유지한다.

> **가상 주소 = 세그먼트 번호 + 세그먼트 내 offset**
> **순방향 세그먼트: offset < 크기 검사 후 PA = base + offset**

- 스택처럼 역방향으로 자라는 영역은 성장 방향을 반영한 오프셋과 범위 검사를 사용한다.
- 읽기 전용 코드 영역을 여러 프로세스가 같은 물리 영역에 매핑하면 안전하게 공유할 수 있다.

방식	장점	핵심 한계
주소 공간 전체의 base/bound	단순하고 빠른 변환·보호	주소 공간 내부의 미사용 영역까지 물리 공간으로 확보
세그멘테이션	사용한 논리 영역만 배치, 영역별 공유·보호	가변 크기 연속 영역 때문에 외부 단편화 발생

외부 단편화는 빈 공간 합계가 충분해도 필요한 연속 구간이 없는 현상이다. 물리 세그먼트를 이동·압축할 수 있지만 복사 비용이 크다.

11 / 핵심 개념

빈 공간 관리

17장 | 요청을 빠르게 처리하면서 단편화를 줄이는 방법

단편화	의미	예
내부 단편화	할당한 블록 안에서 사용하지 못한 공간	7 KiB 요청에 8 KiB 블록 할당
외부 단편화	빈 공간이 흩어져 큰 연속 요청을 충족하지 못함	10+10 KiB가 떨어져 있어 15 KiB 요청 실패

기본 메커니즘

- **분할(splitting)**: 큰 빈 블록에서 요청량을 떼고 나머지를 빈 공간으로 유지한다.
- **병합(coalescing)**: 해제된 블록과 주소상 인접한 빈 블록을 합친다.
- **헤더**: 크기 등 관리 정보를 저장해 free가 포인터만으로 해제 크기를 알 수 있게 한다. 요청 크기 외에 헤더·정렬 비용도 고려한다.

할당 정책 비교

정책	선택 규칙	주의점
Best fit	요청을 수용하는 가장 작은 블록	검색 비용과 작은 잔여 조각 발생
Worst fit	가장 큰 블록	큰 조각을 남기려 하지만 항상 좋은 결과는 아님
First fit	순서대로 보아 처음 맞는 블록	검색이 빠를 수 있으나 앞부분에 단편 집중
Next fit	직전 탐색 위치에서 이어 탐색	탐색을 분산. 리스트 순서에 영향받음

빈 블록 순서가 10·30·20이고 요청이 15라면 Best fit은 20, Worst fit과 First fit은 30을 고른다. 헤더·정렬 비용을 생략한 예다.

개선 방법

크기별 리스트는 자주 쓰는 크기를 빠르게 할당한다. **슬랩**은 특정 객체를 재사용해 초기화 비용을 줄인다. **버디**는 2의 거듭제곱 크기로 분할하고 짝이 비었을 때 재귀 병합한다. 버디는 병합이 쉽지만 내부 단편화가 생긴다.

일반 C 힙 할당기는 이미 사용자에게 준 포인터를 자유롭게 옮길 수 없으므로, 물리 세그먼트 압축과 같은 방식으로 해결하기 어렵다.

12 / 핵심 개념

페이징과 페이지 테이블

18장 | 고정 크기 단위로 나누고 물리 배치를 분리

가상 페이지(page)와 **물리 프레임(frame)**을 같은 고정 크기로 나눈다. 가상 페이지는 RAM의 임의의 빈 프레임에 배치할 수 있다. 프로세스별 페이지 테이블은 VPN을 PFN으로 연결한다.

> **VA = VPN × 페이지 크기 + offset**
> **PA = PFN × 페이지 크기 + offset**
> **offset은 주소 변환 전후 동일하다.**

주소 변환 계산

32비트 VA, 페이지 4 KiB=2^{12}이면 offset 12비트, VPN 20비트다. VA=0x1234는 VPN=1, offset=0x234. VPN 1이 PFN 7에 매핑되면 PA=**0x7234**다.

PTE에 들어가는 정보

정보	의미
PFN	RAM에 있는 물리 프레임 번호
valid	프로세스 주소 공간에서 유효한 페이지인지(교재의 추상 모델)
protection	읽기·쓰기·실행 및 사용자/커널 접근 권한
present	현재 물리 메모리에 있는지
dirty / accessed	내용이 수정됐는지 / 접근된 적이 있는지

페이징이 해결한 문제와 남은 문제

- 같은 크기의 프레임에 분산 배치하므로 기본 페이지 배치에서 외부 단편화를 없앤다. 마지막 페이지 등의 내부 낭비는 남는다.
- 선형 페이지 테이블은 커질 수 있다. 32비트·4 KiB·PTE 4바이트면 2^{20} × 4 = **4 MiB/프로세스**다.
- TLB가 없으면 단순 데이터 접근도 PTE 읽기와 실제 데이터 읽기를 거쳐 추가 비용이 생긴다. 다음 두 장이 속도와 공간 문제를 해결한다.

valid·present 등은 의미를 구분하기 위한 교재 모델이다. 실제 CPU의 PTE 비트 구성과 비트 이름이 항상 일대일로 대응하는 것은 아니다.

13 / 핵심 개념

TLB: 더 빠른 주소 변환

19장 | 자주 쓰는 VPN → PFN 매핑을 CPU 가까이에 저장

> **TLB(Translation Lookaside Buffer)**는 주소 변환 정보의 캐시다.
> 사용자 데이터 자체를 저장하는 데이터 캐시와 역할이 다르다.

상황	동작
TLB hit	권한을 확인하고 PFN과 offset을 합쳐 물리 주소 구성
TLB miss, 페이지는 RAM에 있음	페이지 테이블 검색 → 변환을 TLB에 저장 → 접근 재시도
페이지가 RAM에 없음	페이지 폴트 → 운영체제가 반입·할당 등 처리

지역성과 TLB 도달 범위

- **공간 지역성**: 같은 페이지의 연속 배열 원소는 처음 한 번의 미스 후 같은 변환을 재사용한다.
- **시간 지역성**: 가까운 시점에 같은 페이지를 다시 사용하면 기존 변환을 재사용한다.

> **TLB 도달 범위(reach) = 엔트리 수 × 페이지 크기**
> **64개 × 4 KiB = 256 KiB / 64개 × 2 MiB = 128 MiB**

도달 범위는 같은 페이지 크기의 엔트리와 충분한 연관도를 가정한 용량 계산이다. 실제 히트율은 접근 패턴과 충돌에도 좌우된다.

미스 처리와 문맥 교환

하드웨어 관리 TLB는 하드웨어가 테이블을 탐색한다(교재의 x86 예). 소프트웨어 관리 TLB는 운영체제 핸들러가 변환을 넣는다. 미스 처리 후에는 원래 접근을 다시 실행해야 한다.

프로세스가 바뀌면 같은 VPN이 다른 PFN을 뜻할 수 있다. **TLB 비우기**로 이전 변환을 무효화하거나, **ASID**로 주소 공간을 구분해 여러 프로세스의 변환을 함께 보관한다.

비트 구분: TLB의 valid는 캐시 항목의 유효성이다. PTE의 valid는 주소 공간 내 매핑의 유효성이다. TLB가 비었다고 페이지 자체가 잘못된 주소인 것은 아니다.

14 / 핵심 개념

더 작은 페이지 테이블

20장 | 희소한 주소 공간을 큰 배열 전체로 표현하지 않는다

방법	공간 절약 방식	대가
큰 페이지	페이지 수와 PTE 수 감소	내부 단편화 증가 가능
세그먼트+페이징	코드·힙·스택별 필요한 테이블만 유지	영역 관리가 복잡하고 테이블 할당의 문제가 남음
다단계 테이블	하위 매핑이 없는 테이블 페이지는 생성하지 않음	TLB 미스 때 여러 단계 탐색
역 페이지 테이블	시스템의 물리 프레임당 항목 하나	프로세스+VPN 검색이 필요해 해시 등 사용

다단계 테이블의 핵심

VPN을 여러 인덱스로 나눈다. 페이지 디렉터리의 항목이 다음 테이블 페이지를 가리키고, 마지막 PTE가 데이터의 PFN을 제공한다. 미사용 주소 영역에는 하위 테이블을 두지 않는다.

> **32비트·4 KiB·PTE/PDE 4바이트의 2단계 예**
> **VA = 디렉터리 인덱스 10비트 | 테이블 인덱스 10비트 | offset 12비트**

테이블 한 페이지에는 4096/4=1024개 PTE가 들어간다. 최상위 디렉터리는 4 KiB다. 하위 테이블 2개만 필요하면 전체는 **12 KiB**이고, 모든 하위 테이블이 필요하면 약 **4 MiB+4 KiB**다.

시간과 공간의 절충

- TLB 히트에서는 다단계 탐색을 건너뛴다. 공간 절약의 대가는 주로 TLB 미스 때 나타난다.
- n단계 테이블이면 단순 모델에서 TLB 미스 시 테이블 n번과 데이터 1번의 접근이 필요하다. 캐시 효과는 별도로 고려한다.
- 페이지 단위로 테이블을 할당하므로 거대한 연속 물리 공간을 찾을 필요가 없다. 사용 범위가 작을 때 특히 유리하다.

사용자 페이지 테이블 자체를 커널 가상 공간에 배치하고 일부를 스왑하는 설계도 있다. 이 경우 탐색 과정이 더 복잡해진다.

15 / 핵심 개념

스왑과 페이지 폴트

21장 | RAM보다 큰 가상 메모리를 제공하는 메커니즘

RAM을 모든 가상 페이지 중 일부를 담는 캐시로 본다. 익명 페이지는 스왑 공간에 보관할 수 있고, 변경되지 않은 실행 파일·파일 매핑 페이지는 원본 파일에서 다시 가져올 수 있다.

메모리 접근의 분기

확인 결과	의미	후속 동작
주소가 무효	사용할 수 없는 가상 영역	오류 처리·시그널·종료 등
접근 권한 위반	읽기/쓰기/실행 방식이 허용되지 않음	보호 오류 처리
유효·허용·present=1	페이지가 RAM에 있음	변환을 TLB에 넣고 접근 재시도
유효·허용·present=0	합법적인 페이지가 RAM에 없음	페이지 폴트 핸들러가 필요한 페이지 준비

디스크에서 페이지를 가져오는 절차

1. 빈 물리 프레임을 확보한다.
2. 없으면 교체 정책으로 피해 페이지를 정한다.
3. 보존해야 하는 dirty 데이터는 저장 장치에 기록한다.
4. 요청 페이지를 읽는다. I/O 대기 중 다른 프로세스를 실행할 수 있다.
5. PTE의 PFN·present와 관련 변환을 갱신한다.
6. 폴트를 일으킨 명령어를 다시 실행한다.

미리 여유 메모리 확보하기

빈 프레임이 **low watermark** 아래로 내려가면 백그라운드 회수 작업을 시작하고, **high watermark**까지 확보하면 쉰다. 더티 페이지를 묶어 기록하는 클러스터링은 작은 I/O를 반복하는 비용을 줄인다.

이 장의 기본 폴트는 디스크 반입 모델이다. Demand zero·COW처럼 디스크 읽기 없이 처리하는 페이지 폴트도 있다. "폴트=프로그램 버그" 또는 "폴트=항상 디스크 I/O"로 외우지 않는다.

16 / 핵심 개념

페이지 교체 정책

22장 + 보충 | 무엇을 남기고 무엇을 내보낼지 결정

정책	교체 대상	핵심 성질
OPT / MIN	다음 사용이 가장 먼 페이지	미래를 알아야 함. 최소 미스의 비교 기준
FIFO	가장 먼저 들어온 페이지	단순. 프레임을 늘려도 미스가 늘 수 있음
Random	무작위 페이지	이력 유지 비용이 작으나 실행마다 결과 변동
LRU	가장 오래 사용하지 않은 페이지	시간 지역성 이용. 정확한 접근 이력 유지가 비쌈
Clock	참조 비트가 0인 후보	참조 비트로 LRU를 싸게 근사

Clock의 동작

시계 바늘이 가리킨 페이지의 참조 비트가 1이면 0으로 지우고 다음으로 이동한다. 0이면 교체 후보로 선택한다. 쓰기 비용까지 고려하면 최근에 사용하지 않은 **clean 페이지**를 우선할 수 있다.

자주 출제되는 함정

- **Belady 이상 현상**: FIFO에서 프레임 3→4로 늘려도 폴트가 증가할 수 있다. LRU·OPT는 스택 성질을 가져 같은 문제가 없다.
- **순환 스캔**: 캐시보다 조금 큰 범위를 반복하면 LRU도 불리하다. 한 번 훑는 페이지가 자주 재사용하는 페이지를 밀어내지 않도록 탐색 내성(scan resistance)이 필요하다.
- **쓰래싱**: 활발히 쓰는 작업 집합이 RAM을 초과해 계산보다 페이지 반입·반출에 많은 시간을 쓰는 상태다.

> 쓰래싱 대처: 동시에 실행할 작업 수 감소, 작업 집합 고려, 메모리 확보.
> 요구 페이징·선반입(prefetching)·쓰기 묶음도 성능을 좌우한다.

저장 장치가 RAM보다 훨씬 느리면 낮은 미스율도 큰 비용이다. 보충 자료는 SSD로 비용 비율이 달라지면서 교체 정책의 중요성도 달라짐을 설명한다. AMAT 검산은 19쪽에 정리했다.

17 / 핵심 개념

완전한 VM: VAX/VMS

23장 | 제한된 하드웨어에 여러 기법을 결합한 실제 설계

하드웨어 제약과 해법

제약·문제	설계 선택	목적
512바이트의 작은 페이지	프로세스 공간을 P0·P1으로 나눠 각 테이블 관리	사용하지 않는 힙-스택 사이의 테이블 공간 감소
큰 사용자 페이지 테이블	커널 가상 공간에 두고 필요 시 테이블 페이지도 스왑	물리 메모리 사용 감소
reference bit 없음	프로세스별 FIFO + 전역 second-chance 리스트	단순한 정책을 보완하고 메모리 독점 완화
작은 페이지의 비효율적인 I/O	더티 페이지를 묶어서 기록	쓰기 요청 수 감소

교재의 VAX 주소 공간은 사용자 코드·힙의 P0, 스택의 P1, 보호된 커널의 시스템 공간으로 구성된다. 페이지 0을 접근 불가능하게 두면 NULL 역참조를 검출할 수 있다.

상주 집합과 두 번째 기회

프로세스마다 **RSS(상주 집합 크기)** 제한을 두고, 이를 넘으면 해당 FIFO의 오래된 페이지를 전역 clean/dirty 리스트로 옮긴다. 프레임이 다른 용도로 재사용되기 전에 다시 필요해지면 디스크에서 읽지 않고 되찾는다.

필요할 때만 수행하는 최적화

기법	미루는 작업	이득
Demand zero	실제로 접근할 때 프레임 확보·0 초기화	안 쓰는 페이지의 할당·초기화 비용 제거, 이전 데이터 노출 방지
Copy-on-write	두 주소 공간에 읽기 전용으로 공유한 후 쓰는 순간 복사	fork 후 exec처럼 복사할 필요가 사라지는 경우 낭비 감소

참조 비트가 없어도 잠시 접근을 금지하고 첫 접근의 트랩에서 권한을 복원해 사용 이력을 근사할 수 있다. 정확한 원래 권한은 따로 보관해야 하며, 너무 자주 반복하면 비용이 커진다.

18 / 핵심 개념

Linux VM과 보호

23장 보충 | 성능·메모리 배치·보안을 함께 설계

기능	핵심
kmalloc / vmalloc	교재 모델에서 kmalloc은 물리적으로도 연속, vmalloc은 가상 주소상 연속이며 물리 배치는 분산 가능
다단계 페이지 테이블	첨부의 x86-64 예는 주소 인덱스 9+9+9+9비트와 offset 12비트인 4단계·48비트 모델
Huge pages	큰 페이지로 TLB 도달 범위 확대·테이블 감소. 내부 낭비·할당·스왑 비용과 절충
mmap	파일을 가상 주소에 매핑해 포인터 접근. 필요한 부분은 페이지 폴트 때 가져올 수 있음
활성·비활성 리스트	원본의 2Q 설명: 한 번 접근한 페이지와 재사용된 페이지를 구분해 스캔 오염 완화

32비트 3 GiB/1 GiB 분할, pdflush, 2Q 비율 등의 세부 값·명칭은 원본 시대의 설명이다. 핵심은 사용자/커널 보호, 백그라운드 기록, 재사용 여부를 반영한 회수다.

보호 장치가 막는 대상

장치	목적	한계·연결
NX / XD	스택 등 데이터 페이지에서 명령 실행 금지	기존 코드 조각을 재사용하는 ROP까지 모두 막지는 못함
ASLR / KASLR	사용자 / 커널 주소 배치를 무작위화	공격자가 코드·데이터 위치를 예측하기 어렵게 함
KPTI	사용자 테이블의 커널 매핑을 최소화하고 별도 커널 테이블 사용	일부 공격 경로 완화. 테이블 전환 비용이 있고 모든 추측 실행 공격을 해결하지는 못함

버퍼 오버플로는 할당 경계를 넘어 다른 메모리를 덮어쓰는 버그다. **Meltdown·Spectre**는 추측 실행의 캐시 등 미세구조 흔적을 통해 정보가 새는 문제를 보여 준다. 페이지 권한만으로 모든 정보 유출을 설명할 수 없다.

> **연결해서 기억하기**
> 페이지 테이블은 주소 변환뿐 아니라 공유·권한·COW·요구 할당·커널 격리의 기반이다. 실제 VM은 한 알고리즘이 아니라 여러 기법의 결합이다.

19 / 핵심 개념

핵심 공식과 검산

계산형 문제를 풀 때 단위·가정·추가 비용을 먼저 확인

대상	공식
반환·응답 시간	반환 = 완료 - 도착 / 응답 = 첫 실행 - 도착
주소 변환	VPN = floor(VA/P), offset = VA mod P PA = PFN×P + offset
페이지 테이블 크기	선형 테이블: (가상 주소 공간 크기/P) × PTE 크기
비트 수	$P=2^p$ 바이트 → offset p비트 a비트 VA → VPN a-p비트
TLB 도달 범위	엔트리 수 × 페이지 크기
Stride	stride = K/티켓; 최소 pass 선택; 실행 후 pass 증가

AMAT: 원본 보충의 개정식

$$AMAT = T_M + P_{miss} \times T_D$$

T_M은 기본 메모리 접근 시간, T_D는 미스 때 추가되는 저장 장치 접근 비용이다. P_{miss}는 백분율 대신 0-1 사이의 확률로 넣는다. 미스에도 기본 메모리 비용을 부담하는 모델이다.

조건: T_M=100 ns, T_D=10 ms	검산 결과
미스율 10% = 0.1	100 ns + 1 ms = **1.0001 ms**
히트율 99.9% → 미스율 0.001	100 ns + 10 µs = **10.1 µs**
둘의 비율	1.0001 ms / 10.1 µs **99.02배**

TLB 평균 시간의 단순 모델

TLB 조회 t, RAM 접근 m, 히트율 h, 선형 테이블·페이지 폴트 없음·캐시 효과 없음이면 **EAT = h(t+m)+(1-h)(t+2m)**. 다단계 n단계 미스에서는 t+(n+1)m을 사용한다. 문제의 조회 중첩·캐시 가정이 다르면 식도 바뀐다.

단위: 1 KiB=2^{10} B, 1 MiB=2^{20} B, 1 GiB=2^{30} B. 1 ms=1,000 µs=1,000,000 ns. 원본의 KB/MB 표기는 이진 단위 계산에서 KiB/MiB로 명확히 썼다.

20 / 핵심 개념

마지막 복습: 꼭 구분할 것

정의와 원인을 한 문장으로 설명할 수 있으면 핵심을 이해한 것

비교	구분 기준
프로그램 / 프로세스	저장된 코드 / 실행 상태를 가진 인스턴스
Ready / Blocked	CPU를 기다림 / I/O 등 사건을 기다림
fork / exec	새 프로세스 생성 / 현재 프로그램 이미지 교체
정책 / 메커니즘	선택 규칙 / 선택을 수행하는 구현
응답 / 반환 시간	첫 실행까지 / 완료까지
MLFQ / 비례 배분	관측한 행동으로 응답·반환 개선 / 정해진 CPU 몫 배분
세그멘테이션 / 페이징	논리 영역별 가변 크기 / 고정 크기 페이지
내부 / 외부 단편화	할당 안의 낭비 / 빈 공간의 분산
TLB 미스 / 페이지 폴트	주소 변환 캐시에 없음 / 페이지 상태·권한 처리가 필요
PTE valid / present	가상 공간에서 사용 가능한 매핑 / 지금 RAM에 있음
LRU / Clock	정확한 최근 사용 순서 / 참조 비트로 근사
Demand zero / COW	첫 사용 때 0 초기화 / 첫 쓰기 때 복사

스스로 확인할 네 가지

- 타이머 인터럽트가 없다면, 양보하지 않는 프로세스로부터 OS가 CPU를 회수할 수 있는가?
- TLB 미스인데 페이지가 RAM에 있는 경우와, 디스크에 있는 경우를 각각 설명할 수 있는가?
- 프레임을 늘리면 항상 페이지 폴트가 줄어드는가? FIFO와 LRU의 차이를 말할 수 있는가?
- fork 직후 exec할 자식의 메모리를 전부 복사하는 대신 COW를 쓰면 왜 유리한가?

학습 순서: 1쪽으로 전체 연결 → 3-8쪽 CPU → 9-18쪽 메모리 → 19쪽 계산 → 20쪽 비교. 세부 근거와 긴 예제는 2쪽의 원본 쪽수로 찾아본다.